PID Control, from zero to working code

Physical AI intern assignment · Modern Monk Labs · Maharishi M

> **Key words. Setpoint**: target value · **Error**: target – actual · **Plant**: the machine being controlled · **Gains** K_p, K_i, K_d: how strongly each part reacts · **Overshoot**: going past the target · **Step response**: how the system moves when the target suddenly changes.
> **Code:** https://github.com/Rishiey-txt/pid-control-assignment (Python, NumPy + Matplotlib; `python run_all.py` recreates every graph).

1. The basic idea

Imagine standing in a shower that is too cold. You feel the water, compare it with the temperature you want, and turn the knob towards hot. If it is *much* too cold you turn the knob a lot, and if it is only a little cold you turn it a little. If it has been slightly cold for a long time, you keep nudging the knob until the cold feeling is gone. If you notice the water warming up quickly, you ease off early so you don't get scalded. Then you feel the water again and repeat.

That loop of **measure** → **compare** → **correct** → **repeat** is feedback control, and the three habits above are exactly the **P**, **I** and **D** of a PID controller. A controller never needs to know in advance how much force is "right". It keeps shrinking the gap (the **error**) between the **setpoint** (target) and the measured value, many times per second. Because it always reacts to what really happened, it also corrects surprises such as a push, a heavier load or a weaker spring.

The feedback loop: measure, compare, correct, repeat

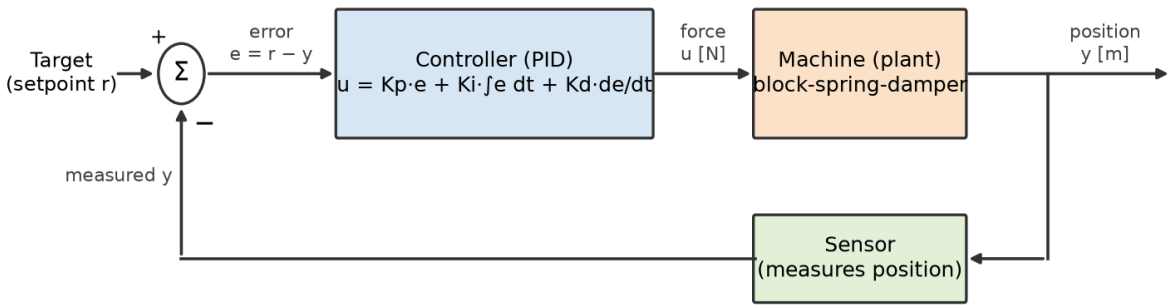


Figure 1. The feedback loop. The sensor's reading is subtracted from the target to give the error; the PID turns the error into a force; the machine moves; the sensor measures again.

2. The PID formula

$$u(t) = K_p\, e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d \frac{de(t)}{dt}, \qquad e(t) = r(t) - y(t)$$

symbol	meaning	unit (our test system)
r(t)	setpoint, the target position	m
y(t)	measured value, the actual position	m
e(t)	error = r – y, how far we still are from the target	m
u(t)	controller output, the force sent to the motor	N
K_p	proportional gain: newtons per metre of error	N/m
K_i	integral gain: newtons per (metre × second) of accumulated error	N/(m·s)
K_d	derivative gain: newtons per (metre/second) of error change	N·s/m
t, τ, T_s	time, integration variable, controller sample time	s

- **P, the present error.** Push in proportion to how far away we are right now. Large error → large push. P alone fades out as the error shrinks, which is why it usually stops a little short (Section 5).
- **I, the past error.** Add up the error over time. Even a tiny error that lasts a long time builds up a large push, so I keeps working until the error is *exactly* zero. It is the only part that can produce a steady force with zero error.
- **D, how fast the error is changing.** If the error is shrinking fast we are about to overshoot, so D pushes back early, like a brake. It acts like extra damping (a shock absorber).

Engineers often write the same controller as $u = K_p(e + (1/T_i)\int e\,dt + T_d\,de/dt)$, where $T_i = K_p/K_i$ (integral time) and $T_d = K_d/K_p$ (derivative time). Section 8 uses this form.

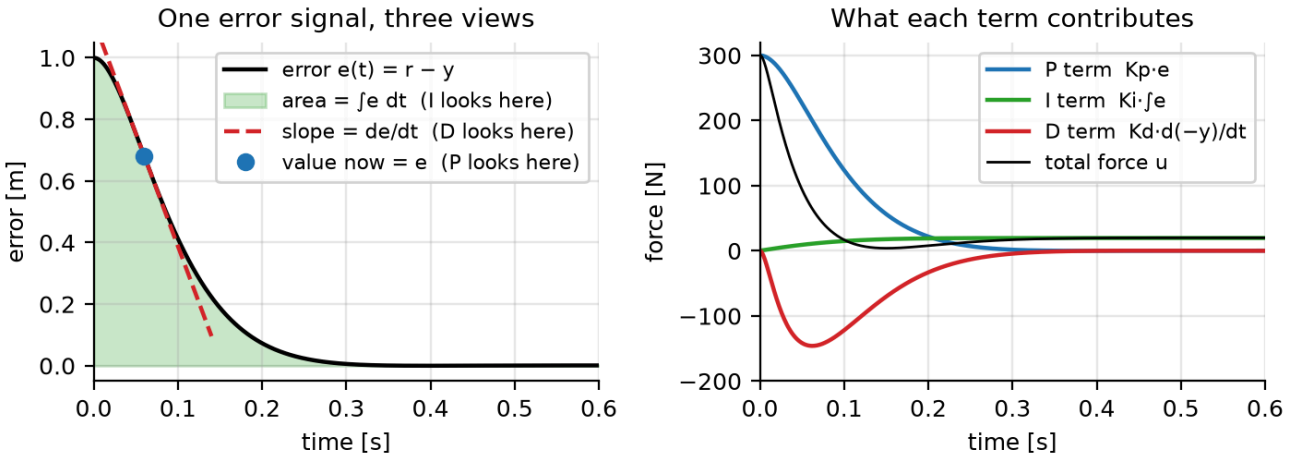


Figure 2. Left: P uses the current value of the error, I uses the area under it, D uses its slope. Right: the force from each term during a step with the final gains of Section 6; I slowly takes over the 20 N needed to hold the block.

The step-by-step version a computer runs

A microcontroller can't integrate or differentiate continuously. Every T_s seconds (here T_s = 1 ms) it replaces the integral with a running sum and the derivative with a difference:

$$e_k = r_k - y_k, \quad I_k = I_{k-1} + K_i\, e_k\, T_s, \quad D_k = K_d\,\frac{e_k - e_{k-1}}{T_s}, \quad u_k = K_p e_k + I_k + D_k$$

```
every Ts seconds:
    error    = setpoint − read_sensor()
    integral = integral + Ki ∗ error ∗ Ts          # running sum   (I)
    deriv    = Kd ∗ (error − prev_error) / Ts      # rate of change (D)
    u        = Kp ∗ error + integral + deriv
    send_to_motor(u);  prev_error = error
```

Section 7 shows why this naive version needs four small changes before it is safe on real hardware.

3. The test system: block on a spring with a damper

A 1 kg block is attached to a wall by a spring (k = 20 N/m) and a damper (c = 10 N·s/m), like one corner of a car suspension. A motor pushes it with force F. Newton's second law (mass × acceleration = sum of forces) gives

$$m\ddot{x} + c\dot{x} + kx = F \;\Rightarrow\; \ddot{x} + 10\dot{x} + 20x = F$$

$$G(s) = \frac{X(s)}{F(s)} = \frac{1}{s^2 + 10s + 20}$$

Without a controller the system is **over-damped** (damping ratio $\zeta = c/(2\sqrt{(km)}) = 1.12 > 1$): its poles are at s = -2.76 and -7.24, so it creeps to rest without bouncing, in about $4/2.76 \approx 1.5$ s.

Why a constant push is needed. When the block is resting at 1 m, velocity and acceleration are zero, so the equation leaves kx = F: the spring pulls back with **20 N/m × 1 m = 20 N**, and the motor must keep pushing 20 N forever. Stop pushing and the spring drags the block back to 0.

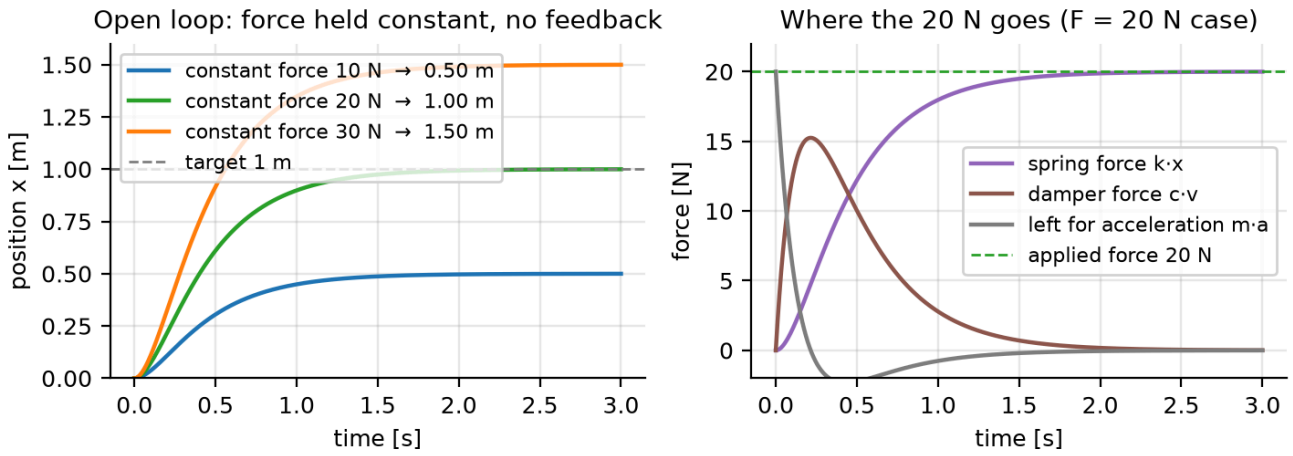


Figure 3. Left: holding a constant force F gives a final position F/k (0.05 m per newton), so exactly 20 N is needed for 1 m. Right: once the block stops, the spring carries the whole 20 N and the damper carries nothing.

This sets the main challenge. The controller must output **20 N while the error is zero**. All simulations step the plant exactly between controller updates (zero-order hold, computed with the matrix exponential), with the controller running at 1 kHz unless stated otherwise.

4. How to judge a result

We judge every controller by its **step response**: the target jumps from 0 to 1 m and we watch what happens. Four numbers sum it up:

1. **Rise time**: how fast it gets there; time to go from 10 % to 90 % of the final value.
2. **Overshoot**: how far it goes past the final value before coming back, in % of the final value.
3. **Settling time**: how long until it calms down; time after which it stays inside ±2 % of the final value.
4. **Steady-state error**: how far from the target it finally stops; setpoint − final value.

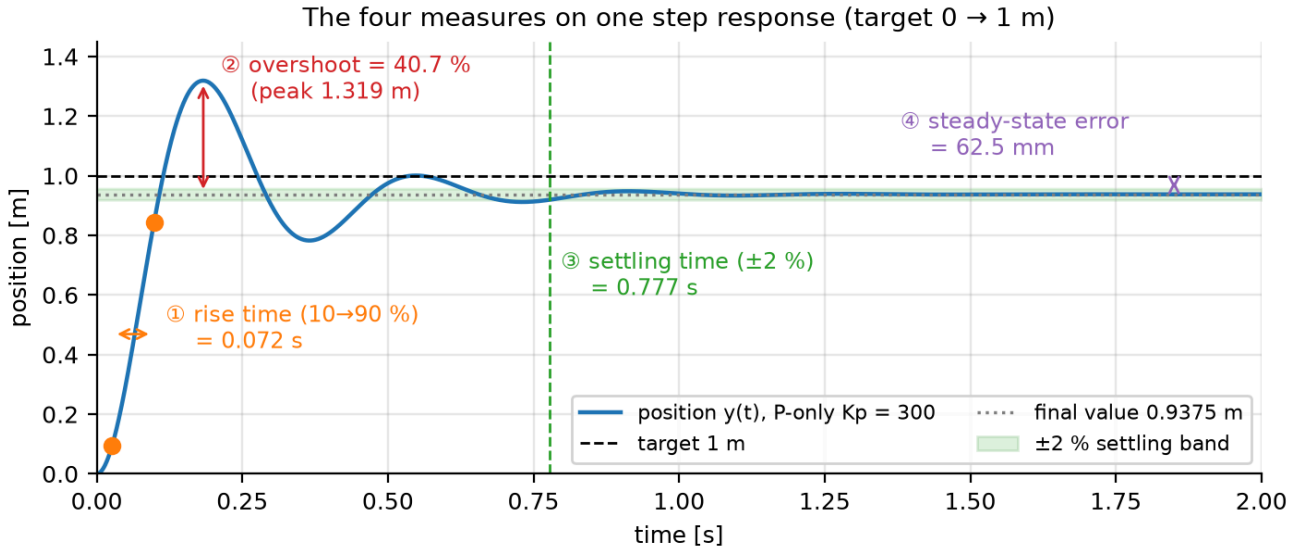


Figure 4. All four measures on a P-only response (K_p = 300): rise 0.072 s, overshoot 40.7 %, settling 0.777 s, and it stops 62.5 mm short of the target.

The measures pull against each other. Pushing harder shortens the rise time but usually increases overshoot; damping more removes overshoot but slows the rise. Tuning means finding a good balance.

5. Each part alone

5.1 P only: fast, but always short of the target

With $u = K_p(r - x)$, the closed loop becomes a new spring-mass-damper whose spring is stiffer by K_p:

$$m\ddot{x} + c\dot{x} + (k + K_p)\,x = K_p\,r$$

Why it stops short. At rest, the motor force $K_p \cdot e$ must balance the spring force $k \cdot x$. If the error were zero the motor would push 0 N and the spring would pull the block back, so the only possible balance has $e > 0$:

$$K_p(1 - x_{ss}) = k\,x_{ss} \;\Rightarrow\; x_{ss} = \frac{K_p}{k + K_p}, \qquad e_{ss} = \frac{k}{k + K_p} = \frac{20}{20 + K_p}\ \text{m}$$

Raising K_p shrinks the error but never removes it. It also lowers the damping ratio $\zeta = c / (2\sqrt{(m(k+K_p))})$, and for $\zeta < 1$ the overshoot of a second-order system is $OS = \exp(-\pi\zeta/\sqrt{(1-\zeta^2)})$. Predictions and simulation agree to 0.1 mm in final value and within 0.9 percentage points in overshoot (the small gap comes from the 1 ms sampling):

K_p [N/m]	final x predicted [m]	final x simulated [m]	error predicted [mm]	error simulated [mm]	ζ	OS predicted [%]	OS simulated [%]
10	0.3333	0.3334	666.7	666.6	0.913	0.1	0.1
50	0.7143	0.7142	285.7	285.8	0.598	9.6	9.7
100	0.8333	0.8333	166.7	166.7	0.456	20.0	20.2
300	0.9375	0.9375	62.5	62.5	0.280	40.1	40.7
500	0.9615	0.9615	38.5	38.5	0.219	49.4	50.3

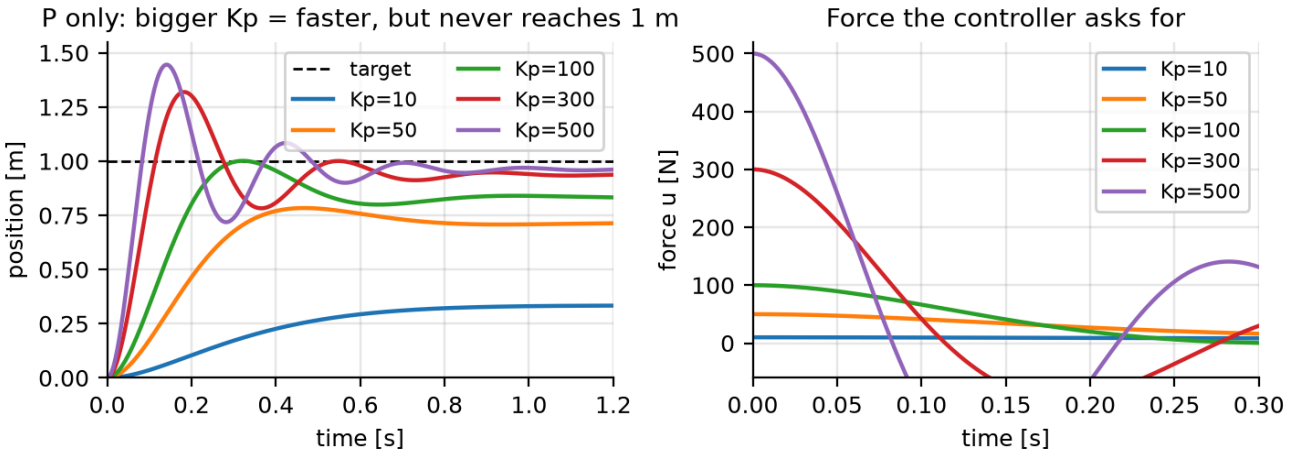


Figure 5. P only. Raising K_p from 10 to 500 cuts the steady-state error from 667 mm to 38.5 mm, but overshoot grows from 0.1 % to 50.3 % and the initial force reaches 500 N.

5.2 Adding I: the offset disappears

The integral keeps growing as long as any error remains, so the loop can only come to rest when the error is exactly zero. At that point the I term alone supplies the 20 N the spring needs (right panel). With K_p = 30 the error drops from 400 mm to 0.0 mm. Too little K_i (30) is very slow (settling 3.81 s). Too much (150) overshoots by 27.8 %, because the I term keeps growing during the whole rise and has to be "unwound" by going past the target.

K_i (K_p = 30)	rise [s]	overshoot [%]	settling [s]	ss-error [mm]
0	0.303	4.4	0.844	400.0
30	1.689	0.1	3.805	6.5
70	0.407	1.3	0.616	0.0
150	0.272	27.8	1.644	-0.0

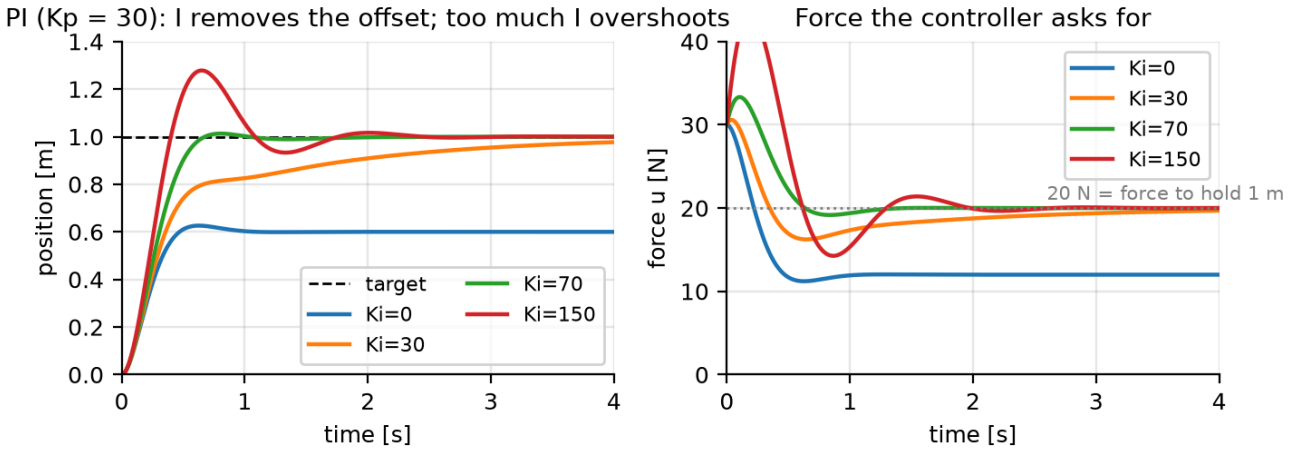


Figure 6. PI control. Any $K_i > 0$ drives the error to zero and the force to the 20 N holding value; $K_i = 70$ does it fastest without real overshoot.

5.3 Adding D: less bouncing

D adds a force $-K_d \cdot \dot{x}$ that opposes velocity, exactly like a bigger damper (c becomes c + K_d):

$$m\ddot{x} + (c + K_d)\,\dot{x} + (k + K_p)\,x = K_p\,r \quad \Rightarrow \quad \zeta = \frac{c + K_d}{2\sqrt{m(k + K_p)}}$$

With $K_p = 300$, ζ rises from 0.28 to 0.84 as K_d goes from 0 to 20, and the overshoot falls from 40.7 % to 0.7 %. Settling time drops by 71 %. Too much D ($K_d = 40$, ζ = 1.4) makes it sluggish again. D is zero once the block stops, so the 62.5 mm offset is unchanged. Predicted and simulated overshoot agree closely:

K_d (K_p = 300)	ζ	OS predicted [%]	OS simulated [%]	rise [s]	settling [s]
0	0.280	40.1	40.7	0.072	0.777
5	0.419	23.4	23.8	0.083	0.469
10	0.559	12.0	12.1	0.098	0.326
20	0.839	0.8	0.7	0.146	0.229
40	1.398	0.0	0.0	0.300	0.547

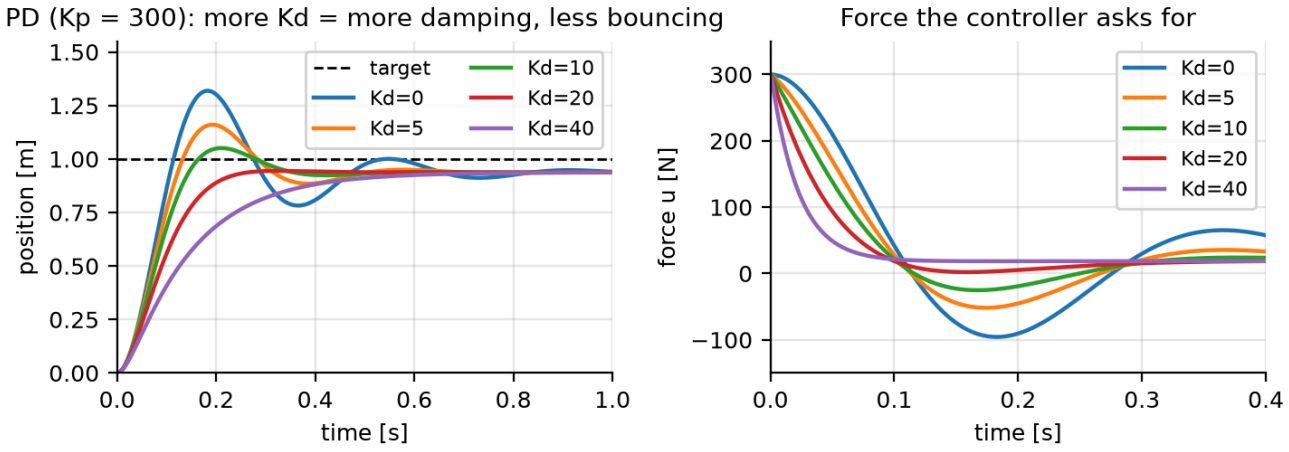


Figure 7. PD control. Extra damping from K_d removes the bounce without slowing the rise much, but the 62.5 mm offset of P control remains.

6. All together

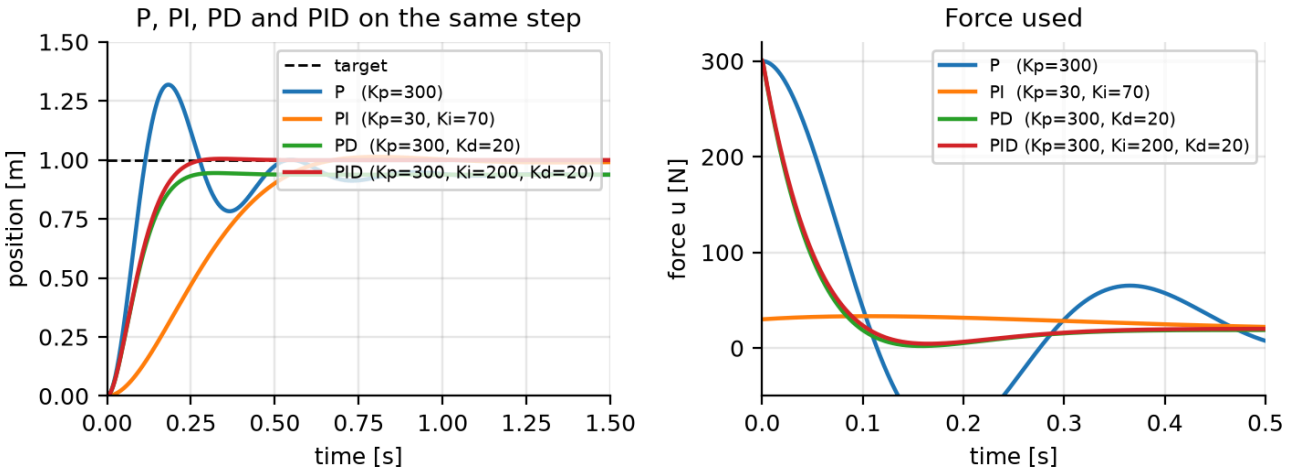


Figure 8. The best P, PI, PD and PID from Section 5 on one step. Only PI and PID reach the target; PID is the only one that is both fast and on target.

controller	gains	rise [s]	overshoot [%]	settling [s]	ss-error [mm]	peak force [N]
P	Kp=300	0.072	40.7	0.776	62.5	300
PI	Kp=30, Ki=70	0.407	1.3	0.616	0.0	33
PD	Kp=300, Kd=20	0.146	0.7	0.229	62.5	300
PID	Kp=300, Ki=200, Kd=20	0.153	0.5	0.242	0.1	300

Choosing the final gains. I followed the hand-tuning recipe of Section 8. $K_p = 300$ gives a fast rise. $K_d = 20$ brings ζ to 0.84, which removes the bounce (overshoot 40.7 % → 0.7 %). Finally, K_i is the smallest value that removes the offset quickly without bringing overshoot back:

K_i (K_p=300, K_d=20)	rise [s]	overshoot [%]	settling [s]	ss-error [mm]
0	0.146	0.7	0.229	62.5
100	0.160	0.0	0.771	7.2
200	0.153	0.5	0.242	0.1
400	0.139	5.8	1.025	-0.1
800	0.121	15.0	0.915	-0.0

$K_i = 100$ is too slow: it creeps the last 7 mm and needs 0.77 s to settle. $K_i \geq 400$ overshoots (5.8 % and more). **$K_i = 200$** settles in 0.242 s with 0.5 % overshoot.

Final gains: $K_p = 300$ N/m, $K_i = 200$ N/(m·s), $K_d = 20$ N·s/m. Compared with P alone, overshoot drops from 40.7 % to 0.5 %, settling from 0.776 s to 0.242 s, and the 62.5 mm error is gone.

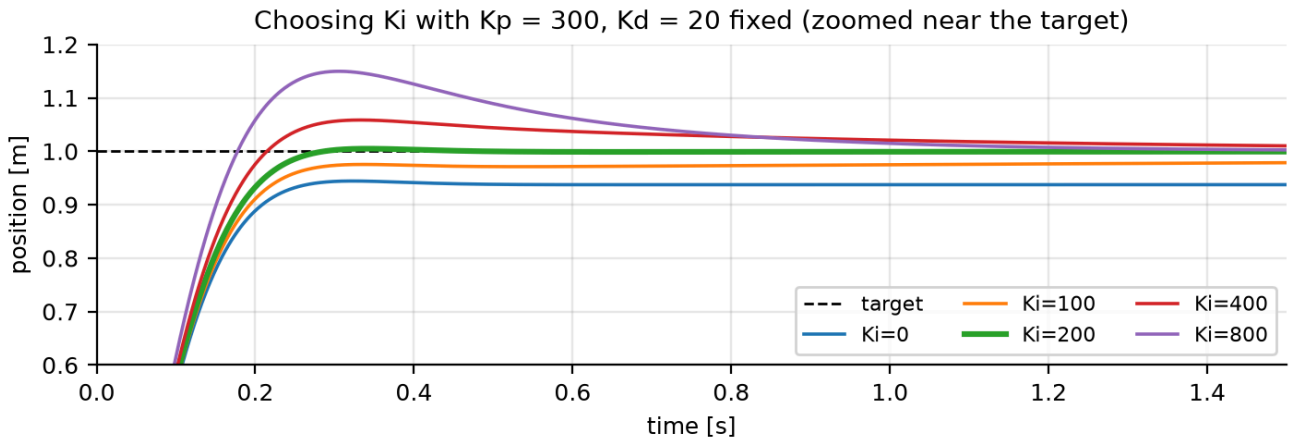


Figure 9. Zoom near the target with K_p and K_d fixed. K_i = 200 (thick line) removes the offset with only 0.5 % overshoot; larger values overshoot more, smaller ones creep.

Catch: these gains ask for a 300 N peak force (K_p × 1 m of initial error). A real motor may not have that. Section 7a shows what happens with only 30 N.

7. Real-world problems and their fixes

All four experiments use the final gains (300 / 200 / 20). Each problem is shown first, then fixed in the controller code (`pidlab/controller.py`).

(a) Force limit of 30 N → integral windup

The PID asks for 300 N at the start but the motor saturates at 30 N, so the block moves slower than the controller "expects". The error stays large for longer, and the I term keeps accumulating. It climbs to 57 N when only 20 N is needed. That stored extra force then pushes the block past the target. **Fix: anti-windup.** *Clamping* stops integrating while the output is saturated and the error would push it further. *Back-calculation* feeds the difference between the requested and the delivered force back into the integrator, bleeding off the excess at rate $K_b(u_{sat} - u)$ with $K_b = K_i/K_p$.

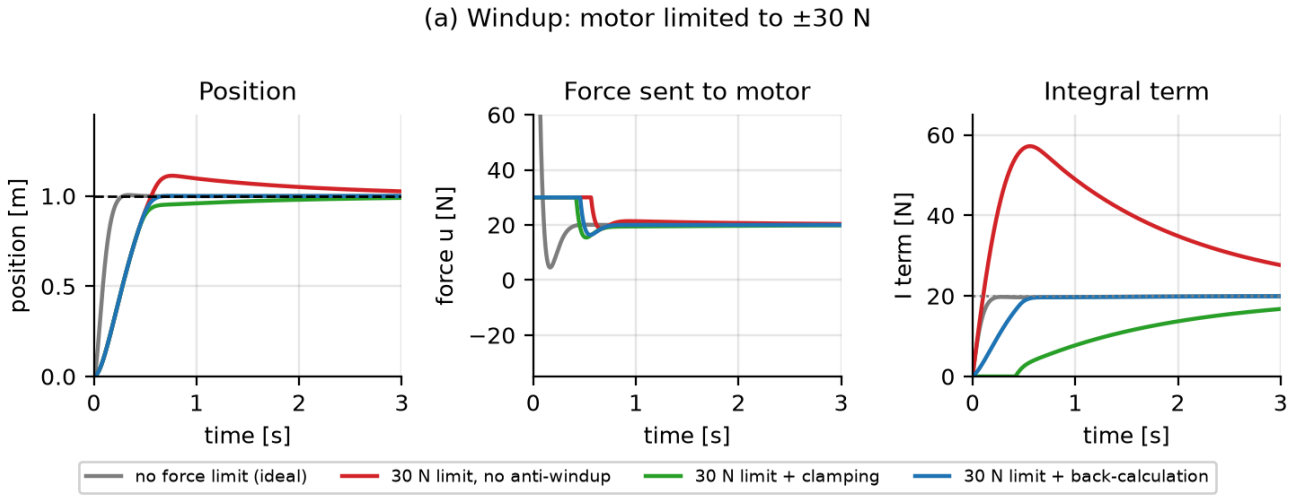


Figure 10. Windup. Without anti-windup the I term (right) grows to almost 3× the 20 N needed and the block overshoots; back-calculation keeps the I term at 20 N and removes the overshoot.

case	rise [s]	overshoot [%]	settling [s]	ss-error [mm]	max I term [N]
no force limit (ideal)	0.153	0.5	0.242	0.0	20.0
30 N limit, no anti-windup	0.395	11.1	3.284	-1.0	57.2
30 N limit + clamping	0.410	0.0	2.040	0.4	19.9
30 N limit + back-calculation	0.396	0.0	0.583	0.0	20.0

Back-calculation is best here: overshoot 11.1 % → 0 %, settling 3.28 s → 0.58 s. Clamping also removes the overshoot, but the I term starts building only after the saturation ends, so the last few millimetres take 2.04 s. No controller can beat the ~0.4 s rise time: that is the physical limit of a 30 N motor against a 20 N spring. Back-calculation is the default in the final code.

(b) Noisy sensor (±1 mm) → jumpy D term

D divides the change in measurement by T_s. Noise that jumps by up to 2 mm between two 1 ms samples looks like a velocity of 2 m/s, and $K_d = 20$ turns it into ±40 N of random force. **Fix: a low-pass filter on the derivative**, $d_k = d_{k-1} + \alpha(\text{raw}_k - d_{k-1})$ with $\alpha = T_s/(\tau + T_s)$, $\tau = 5$ ms (cut-off $1/(2\pi\tau) \approx 32$ Hz, well above the ~3.6 Hz bandwidth of the loop).

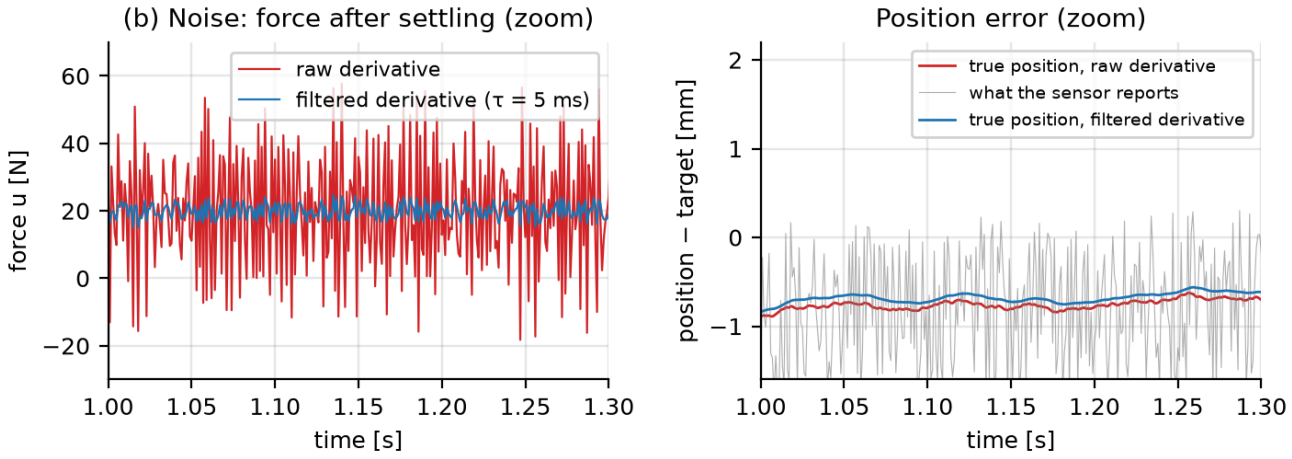


Figure 11. Sensor noise. Left: with a raw derivative the motor force chatters between –20 and +60 N; the filtered version stays within a few newtons of 20 N. Right: the block itself barely moves in either case.

After settling, the force's standard deviation falls from 17.4 N to 2.2 N (87 % less) and the peak-to-peak from 78 N to 11 N, while the step response hardly changes (settling 0.241 s → 0.257 s). The block's position jitter is only about 0.15 mm either way, because the mass filters fast forces. The real damage of a noisy D term is heat, wear and audible buzz in the motor.

(c) Sudden target change → derivative kick

When the target jumps by 1 m in one sample, the error jumps too, and de/dt ≈ 1 m / 0.001 s = 1000 m/s. D outputs $K_d \times 1000$ = **20 000 N** for one sample (simulated: 20000 N). **Fix: take the derivative of the measurement instead of the error**, $D = -K_d \cdot dy/dt$. When the target is constant this is identical to de/dt, but the block's position can't jump (it has mass), so there is no spike.

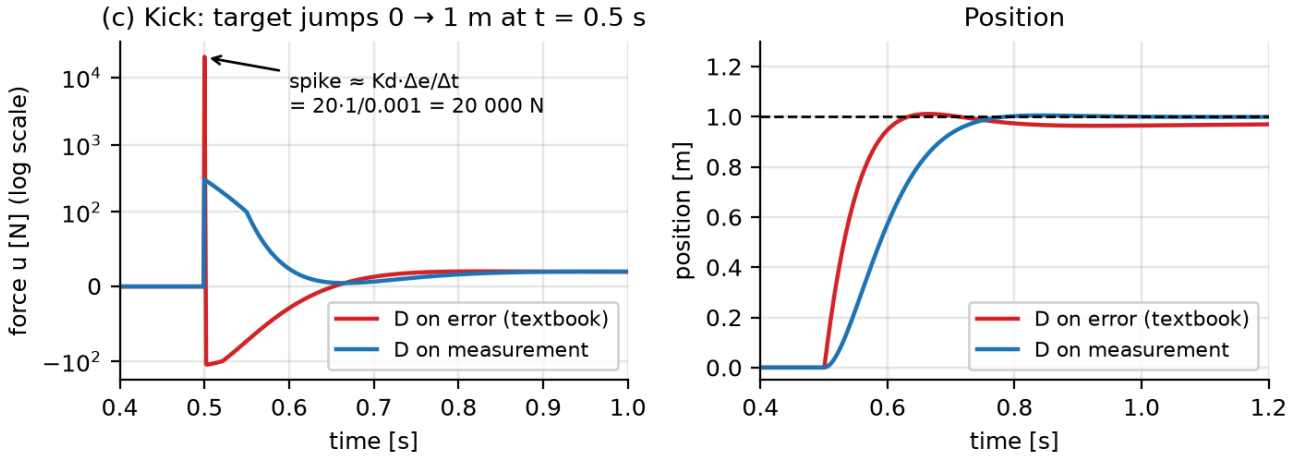


Figure 12. Derivative kick. Left (log scale): D on error produces a 20300 N spike; D on measurement peaks at 300 N (from P alone). Right: the kick throws the block forward.

(d) Controller updates too slowly

Between updates the force is frozen (right panel below), so the controller reacts to old information. This delay of about $T_s/2$ eats into the **phase margin**, the safety margin against oscillation.

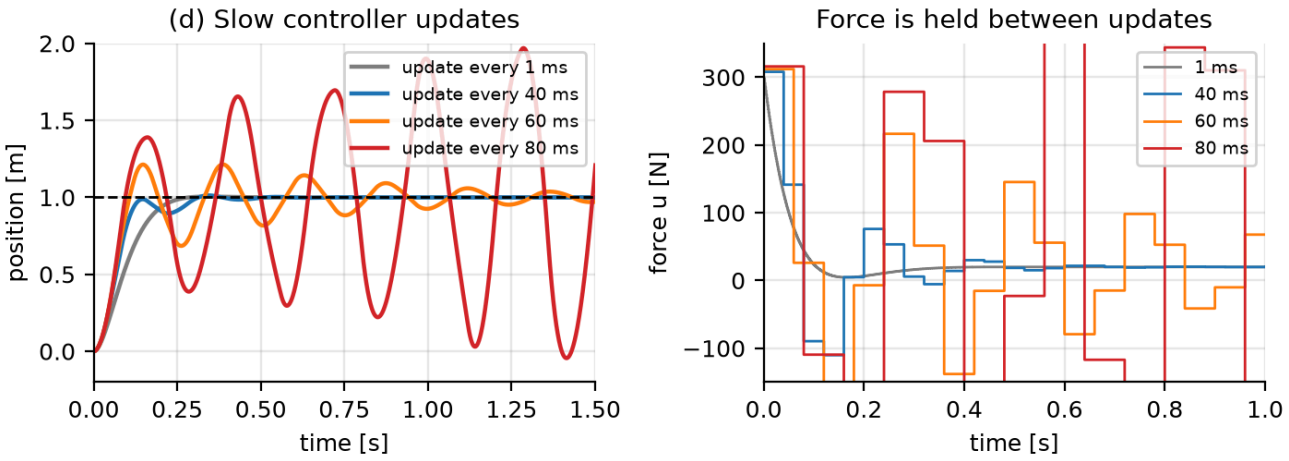


Figure 13. Slow updates. 40 ms still works; at 60 ms the block rings with 21 % overshoot; at 80 ms the oscillation grows and the loop is unstable.

T_s [ms]	max |eigenvalue|	overshoot [%]	settling [s]
1	0.9993	0.5	0.242
20	0.9869	0.0	0.262
40	0.9741	1.3	0.296
50	0.9678	9.0	0.723
60	0.9616	21.4	1.636
70	0.9754	—	slowly decaying
80	1.0453	—	**unstable**

Checking by calculation. The sampled loop (PID code + exactly discretised plant) is a linear recurrence $z_{k+1} = \Phi\, z_k$. It is stable when every eigenvalue of Φ has magnitude below 1. A sweep gives the exact limit $\mathbf{T_s = 73.5\ ms}$, which matches the simulations (70 ms rings for a long time, 80 ms blows up). A quick hand estimate: the continuous loop crosses gain 1 at $\omega_c = 22.4$ rad/s with 81° of phase margin. Sampling plus the backward-difference derivative cost about $\omega_c T_s$ radians, so the limit is $T_s \approx \mathrm{PM}/\omega_c \approx 63$ ms. That is slightly pessimistic but in the right place. **Rule of thumb:** sample at least 10–20× faster than the loop bandwidth ($\omega_c \approx 3.6$ Hz → ≥ 40–70 Hz). We use 1 kHz.

8. Choosing the gains

Tuning by hand

1. **Start safe.** Set $K_i = K_d = 0$, small K_p. Check the sign: the output must move *towards* the target.
2. **Raise K_p** until the response is fast but overshoots clearly (here 300: rise 0.07 s, 41 % overshoot). Back off if the force limit or noise becomes a problem.
3. **Add K_d** until the overshoot is almost gone (aim for $\zeta \approx 0.7$–0.9; here $K_d = 20$ → 0.7 %). Stop before it becomes sluggish.
4. **Add K_i** slowly until the steady-state error disappears in acceptable time. Stop when overshoot starts to return (here 200).
5. **Make it real.** Add the output limit with anti-windup, D filter and D on measurement, then re-check with a disturbance, noise and a different setpoint. Fine-tune by ±20 %.

Ziegler–Nichols on a second system, $G(s) = 1/(s+1)^3$

This plant is three slow lags in series, a common model of thermal or process systems. Following Ziegler–Nichols, I used P only and raised K_p until the output swung with constant amplitude. In code this is a bisection on K_p until a late swing is exactly as large as an early one. Result: $\mathbf{K_u = 7.988}$ and $\mathbf{P_u = 3.630\ s}$. *Check:* the loop is on the edge of stability when $(j\omega+1)^3 + K = 0$. The imaginary part gives $\omega = \sqrt{3}$ and the real part $K = 8$, so $P_u = 2\pi/\sqrt{3} = 3.628$ s. Both match.

Z-N recipe	K_p	T_i	T_d
P	0.5 K_u	—	—
PI	0.45 K_u	$P_u/1.2$	—
PID (classic)	0.6 K_u	$P_u/2$	$P_u/8$

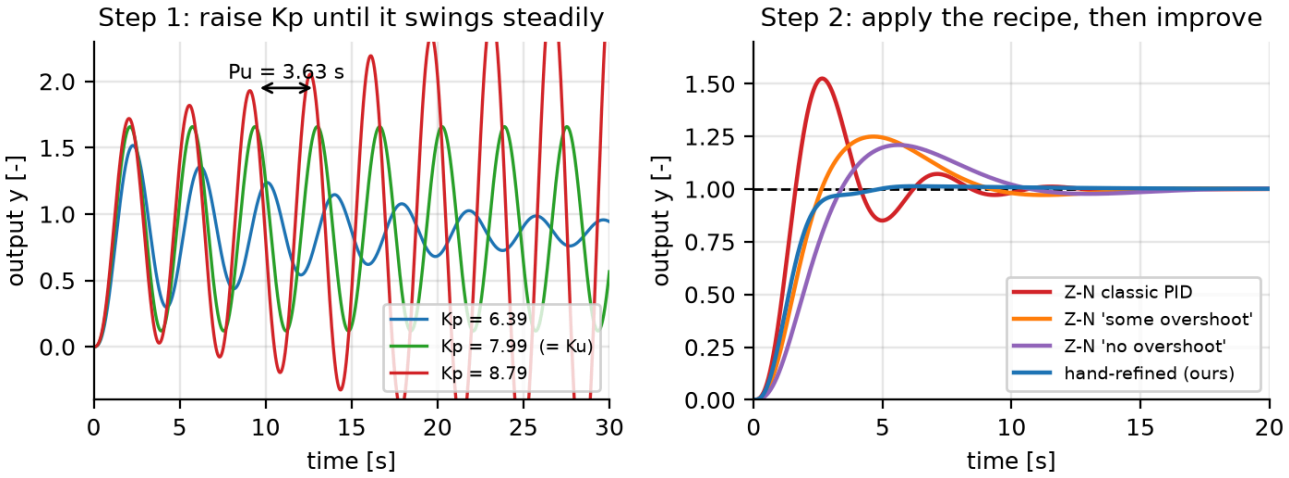


Figure 14. Left: at $K_p = K_u$ the oscillation neither grows nor decays, and its period is P_u. Right: classic Z-N PID overshoots by 52 %; the hand-refined version overshoots by 1.4 % and settles in less than half the time.

tuning	K_p	T_i [s]	T_d [s]	rise [s]	overshoot [%]	settling [s]	ss-error [mm]
Z-N classic PID	4.79	1.81	0.45	0.951	52.4	9.908	-0.0
Z-N 'some overshoot'	2.66	1.81	1.21	1.629	24.9	12.838	0.0
Z-N 'no overshoot'	1.60	1.81	1.21	2.133	20.9	14.013	-0.0
hand-refined (ours)	3.99	5.45	0.91	1.713	1.4	4.004	-0.0

Improving it. Classic Z-N aims for a "quarter-amplitude decay", which is deliberately aggressive: 52.4 % overshoot and 9.9 s settling. The overshoot here comes mostly from the integral filling up during the slow rise. So I made three changes: (1) lowered K_p to 0.5 K_u, (2) tripled T_i to 1.5 P_u (much less integral action) and (3) doubled T_d to $P_u/4$ (more damping). Overshoot fell from 52.4 % to 1.4 % and settling from 9.9 s to 4.0 s. The price was a slower rise (0.95 s → 1.71 s). The published "some/no overshoot" variants still gave 21–25 % here, a reminder that recipes are starting points, not answers.

9. The code

The repository (https://github.com/Rishiey-txt/pid-control-assignment) contains a small library `pidlab/` and one script per section in `experiments/`. `python run_all.py` recreates every figure and number in this document in about 5 s. Only NumPy and Matplotlib are used; the PID logic, the plant simulation (exact zero-order hold using a hand-written matrix exponential) and the performance measures are all written from scratch.

```
class PID:
    def __init__(self, kp, ki=0, kd=0, dt=0.001, u_min=-inf, u_max=inf,
                 anti_windup="backcalc", kb=None, d_filter_tau=0.0, d_on_measurement=True): ...

    def update(self, setpoint, measurement):
        error = setpoint - measurement
        p = self.kp * error                                          # P: present
        d_input = -measurement if self.d_on_measurement else error  # fix (c)
        raw_rate = (d_input - self.prev_input) / dt
        self.d_state += dt / (self.d_filter_tau + dt) * (raw_rate - self.d_state)  # fix (b)
        d = self.kd * self.d_state                                   # D: trend
        i_new = self.integral + self.ki * error * dt                 # I: past
        u_unsat = p + i_new + d
        u = min(max(u_unsat, self.u_min), self.u_max)                # actuator limit
        if self.anti_windup == "clamp" and saturated_in_direction_of(error):
            i_new = self.integral                                    # fix (a), option 1
        elif self.anti_windup == "backcalc":
            i_new += self.kb * (u - u_unsat) * dt                    # fix (a), option 2
        self.integral = i_new
        return u                                                     # fix (d): dt is explicit
```

Abridged; the full file includes first-call handling and parameter checks. Usage: `pid = PID(300, 200, 20, dt=0.001, u_min=-30, u_max=30, d_filter_tau=0.005)`, then call `u = pid.update(target, sensor_reading)` once per sample.

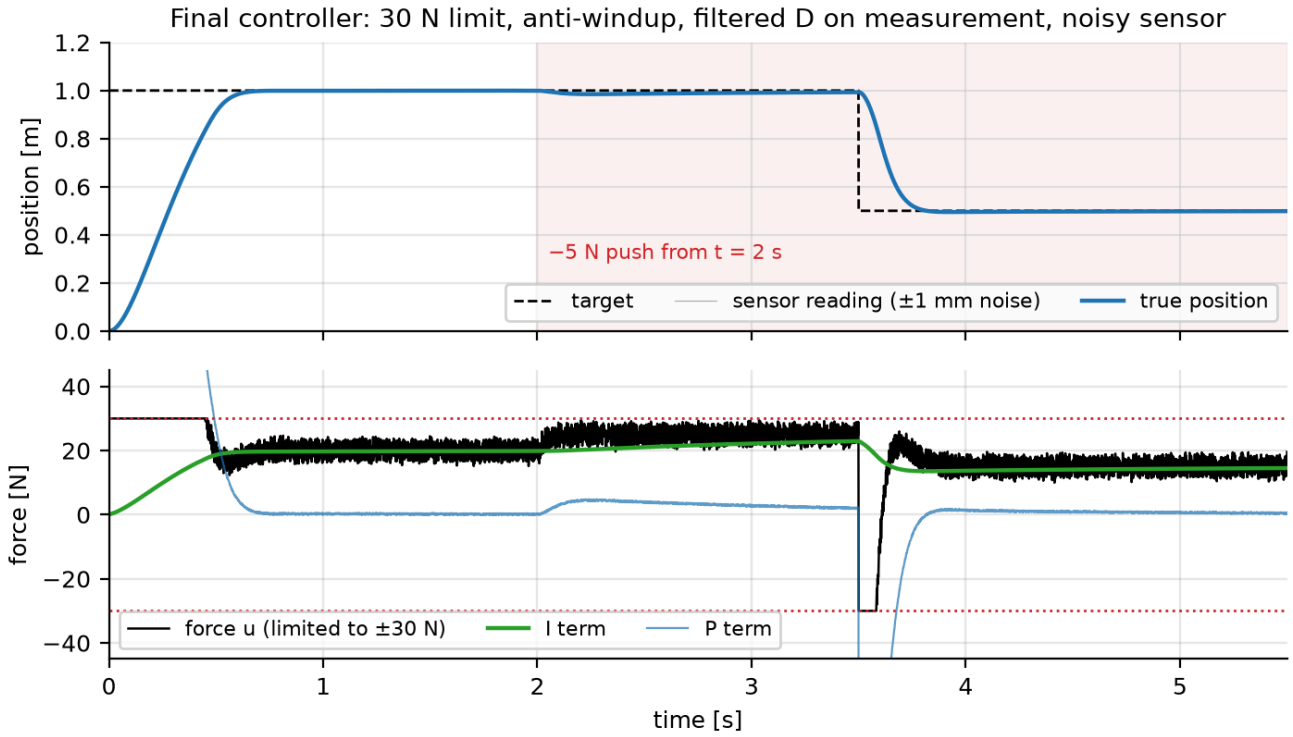


Figure 15. The final controller with every fix on, a 30 N motor and a noisy sensor: step to 1 m, a constant −5 N push at 2 s, then a new target of 0.5 m at 3.5 s.

Results. Step 0 → 1 m: rise 0.396 s, overshoot 0.0 %, settling 0.588 s, final error 0.5 mm, despite the force limit and the noise. The −5 N push only dents the position by 15 mm. The I term then rises from 19.9 N towards 25 N to cancel it, which P and D alone could never do. Step 1 → 0.5 m: overshoot 0.7 %, settling 0.280 s, and no derivative kick at the target change.

10. Real robots & common myths

system	what is controlled	how PID is used
DC motor speed (conveyors, wheels)	shaft speed from an encoder	PI on speed (D is rarely needed because the motor is a first-order system), usually over a fast inner current loop; anti-windup because the voltage saturates.
Quadcopter drone	roll/pitch/yaw angle and rate	Cascade: an outer angle loop (P) sets a target rotation rate for an inner rate PID running at 1–8 kHz. D is heavily filtered because of propeller vibration, and I is held back during flips to avoid windup.
Robot arm joint	joint angle	PID or cascaded position → velocity → current loops in each servo drive, plus a gravity feed-forward term so the I term does not have to carry the arm's weight.
Line-following robot	sideways offset from the line	PD on the position error from an IR sensor array; the output is a difference in wheel speeds. I is often left out because a small offset is acceptable.
Car cruise control	vehicle speed	PI on speed → throttle. I removes the error on hills, and anti-windup is needed when the throttle is fully open on a steep climb.
3D-printer hot end	nozzle temperature	PID with a relay 'auto-tune' that measures the swing, as in Section 8. I removes the offset caused by heat loss to the air.

Myths

- **"Higher gains are always better/faster."** Raising K_p from 100 to 500 raised overshoot from 20 % to 50 % (Fig. 5). High gains ask for forces the motor doesn't have (Fig. 10), and any delay, such as slow sampling, pushes an aggressive loop into oscillation (Fig. 13).
- **"You always need all three terms."** Many good loops are PI (motor speed, cruise control) or PD (line followers). D amplifies noise (Fig. 11), and I adds overshoot and windup. Use only what the problem needs.
- **"The D term predicts the future."** It only extends the current slope. With a noisy sensor it "predicts" noise (±40 N of chatter here); with a target jump it predicts nonsense (a 20 000 N kick).
- **"Ziegler–Nichols gives optimal gains."** It gave 52 % overshoot on $1/(s+1)^3$. It is a quick, aggressive starting point.
- **"PID doesn't need any model or physics."** It can be tuned without one, but knowing the plant explained every result here: the 20 N holding force, why P is short by $20/(20+K_p)$, and where the loop goes unstable.
- **"Once tuned, it stays tuned."** If the mass, friction, battery voltage or load change, the loop behaves differently. Real systems re-tune, add feed-forward, or schedule gains.

11. Reflection: approach, what worked, what didn't, what next

Approach. I built everything bottom-up so every claim could be checked. I wrote a small simulator (exact zero-order-hold plant, PID called at a fixed rate like on a microcontroller) and one metrics function used everywhere. Before trusting any graph, I compared it with a hand calculation. I tuned by hand on the ideal system, then added the real-world limits one at a time, so each fix could be shown against its own problem.

What worked

- Theory and simulation agree: P-only final values to 0.1 mm, overshoot within 0.9 pp, exact stability limit 73.5 ms vs simulation, Z-N K_u 7.99 vs 8.
- Hand tuning (P → D → I) reached 0.5 % overshoot and 0.24 s settling.
- Back-calculation anti-windup: overshoot 11.1 % → 0 % with a 30 N motor.
- D filter: 87 % less force chatter at almost no cost in speed.
- Refining Z-N: overshoot 52 % → 1.4 %.

What didn't work (or only partly)

- **Clamping anti-windup** removed the overshoot but left a slow creep (2.04 s vs 0.58 s), so I switched the default to back-calculation.
- The published Z-N **"some/no overshoot" rules** still overshot by 21–25 % on $1/(s+1)^3$.
- My **quick estimate of the sampling limit** (≈ 63 ms) was 11 ms too pessimistic, so I added the exact eigenvalue check.
- The final gains **ask for 300 N**. With the real 30 N motor the motor sets the speed (rise ≈ 0.4 s), and the gains were never re-tuned for it.

- The model is **idealised**: linear, no friction, backlash or motor dynamics, and the noise is white and uniform.

What I would explore next with more time

- **Design for the real 30 N motor:** add the known 20 N spring force as *feed-forward* (u = k·r + PID) and ramp the target, so the loop does less work and stays out of saturation.
- **Automatic tuning:** minimise a cost (e.g. ITAE plus force and noise penalties) over K_p, K_i, K_d, and compare with hand tuning.
- **Robustness:** sweep mass, spring and delay to map where the chosen gains stay acceptable; report gain and phase margins.
- **Better sensing:** a Kalman filter that estimates velocity, so D works from a clean estimate instead of a filtered difference.
- **Beyond single-loop PID:** cascaded position/velocity loops, setpoint weighting, and a comparison with LQR/MPC.
- **Hardware:** run the same `PID` class on a microcontroller with a real DC motor and encoder, and compare with the simulation.

Notes

Use of AI tools. Claude (Anthropic) helped me structure the code and draft the text. Every number and graph in this document is produced by the code in the repository, and the key results were checked against hand calculations (Sections 5, 7d and 8). **Missing items:** none. **References:** K. J. Åström & R. M. Murray, *Feedback Systems* (2nd ed.); K. J. Åström & T. Hägglund, *Advanced PID Control*; University of Michigan "Control Tutorials for MATLAB & Simulink", PID introduction (same mass-spring-damper example).